AD23532 – Principles of Data Science Lab

Google Colab notebook: Experiments 1–7

Jeyaguru M 2116241801108

```
!pip -q install pandas numpy matplotlib seaborn scikit-learn nltk spacy statsmodels wordcloud
import os, re, warnings, numpy as np, pandas as pd, matplotlib.pyplot as plt, seaborn as sns
warnings.filterwarnings("ignore")
SEED=42
pd.set_option("display.max_columns",100)
```

Shared: load Telco Customer Churn dataset

```
#Jeyaguru M
urls=["https://raw.githubusercontent.com/blastchar/telco-customer-churn/master/WA_Fn-UseC_-Telco-Customer-Churn.csv"]
telco=None
for u in urls:
    try: telco=pd.read_csv(u); break
    except Exception: pass
if telco is None:
    from google.colab import files
    up=files.upload(); telco=pd.read_csv(next(iter(up)))
print(telco.shape); display(telco.head())
```

```
Choose files telco.csv
telco.csv(text/csv) - 202955 bytes, last modified: 30/10/2026 - 100% done
Saving telco.csv to telco.csv
(1500, 21)
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecuri
0	ID00000	Female	0	No	Yes	6	Yes	Yes	Fiber optic	
1	ID00001	Male	0	No	No	34	Yes	No	DSL	
2	ID00002	Male	0	Yes	No	47	Yes	No	Fiber optic	
3	ID00003	Female	0	No	No	4	Yes	No	No	No inter serv
4	ID00004	Female	0	No	No	28	Yes	No	Fiber optic	

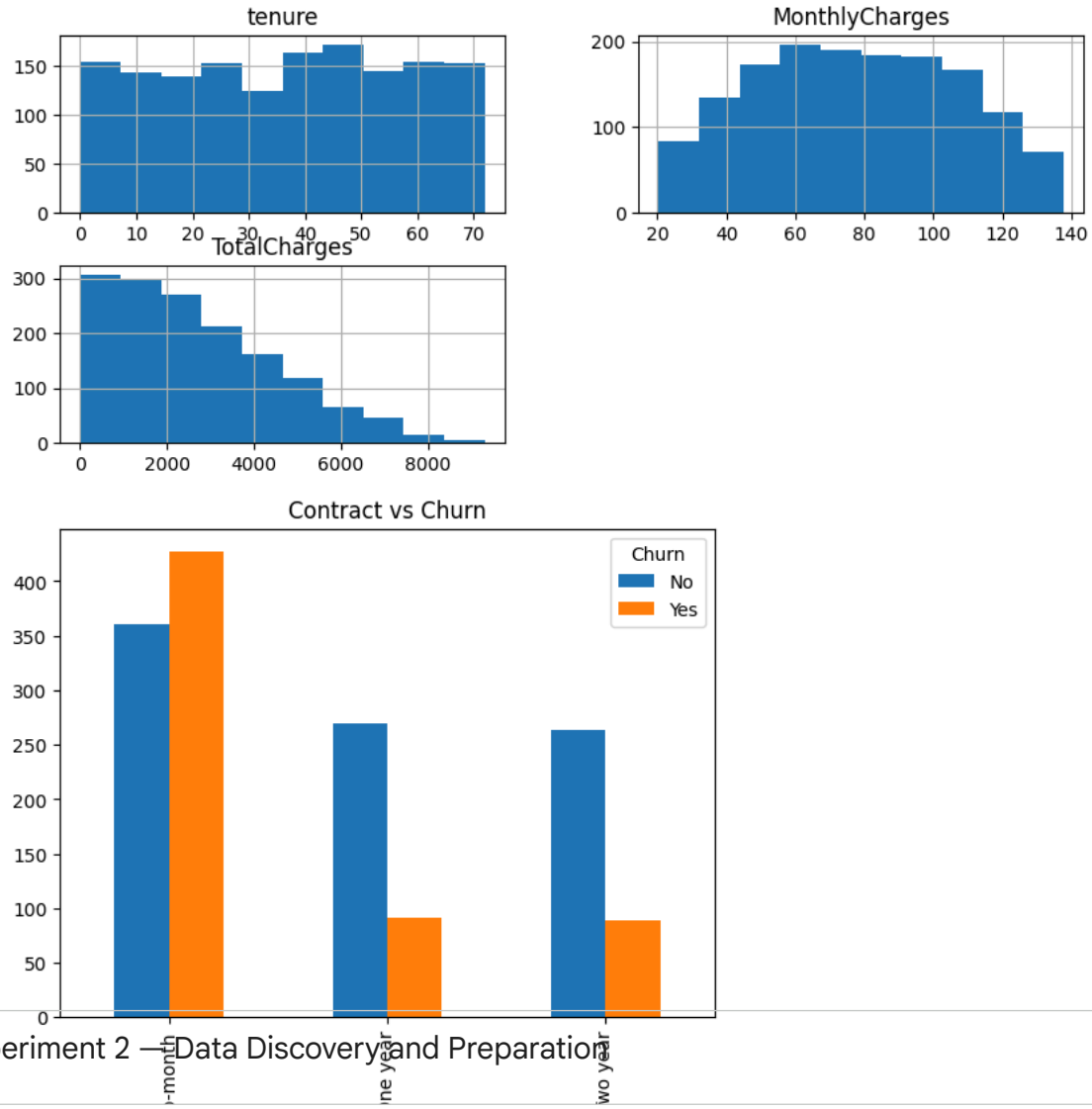
Experiment 1 — Data Preprocessing and Cleaning

```
#Jeyaguru M
df1=telco.copy()
if "TotalCharges" in df1.columns:
    df1["TotalCharges"]=pd.to_numeric(df1["TotalCharges"],errors="coerce")
    df1=df1.drop_duplicates().drop(columns=["customerID"],errors="ignore")
    for c in df1.select_dtypes(include=np.number): df1[c]=df1[c].fillna(df1[c].median())
    for c in df1.select_dtypes(exclude=np.number):
        mode=df1[c].mode()
        df1[c]=df1[c].fillna(mode.iloc[0] if len(mode) else "Unknown")
    plotdf=df1.copy()
    df1=pd.get_dummies(df1,drop_first=True,dtype=int)
    from sklearn.preprocessing import StandardScaler
    cols=[c for c in ["tenure","MonthlyCharges","TotalCharges"] if c in df1]
    if cols: df1[cols]=StandardScaler().fit_transform(df1[cols])
    display(df1.head()); print("Cleaned shape:",df1.shape)
    plotdf[["tenure","MonthlyCharges","TotalCharges"]].hist(figsize=(10,4)); plt.show()
    if "Contract" in plotdf and "Churn" in plotdf:
        pd.crosstab(plotdf["Contract"],plotdf["Churn"]).plot(kind="bar"); plt.title("Contract vs Churn"); plt.show()
    if "Churn" in plotdf and "MonthlyCharges" in plotdf:
        sns.boxplot(data=plotdf,x="Churn",y="MonthlyCharges"); plt.show()
    plt.figure(figsize=(12,8)); sns.heatmap(df1.corr(numeric_only=True),cmap="coolwarm",center=0); plt.show()
    df1.to_csv("Telco_Cleaned_Encoded.csv",index=False)
    print("Saved Telco_Cleaned_Encoded.csv")
else:
    print("Warning: 'telco' DataFrame does not contain Telco Customer Churn data. Columns present are:", list(df1.columns))
```

```
    print("Warning: 'telco' DataFrame does not contain Telco Customer Churn data. Columns present are:", list(df1.columns))
```

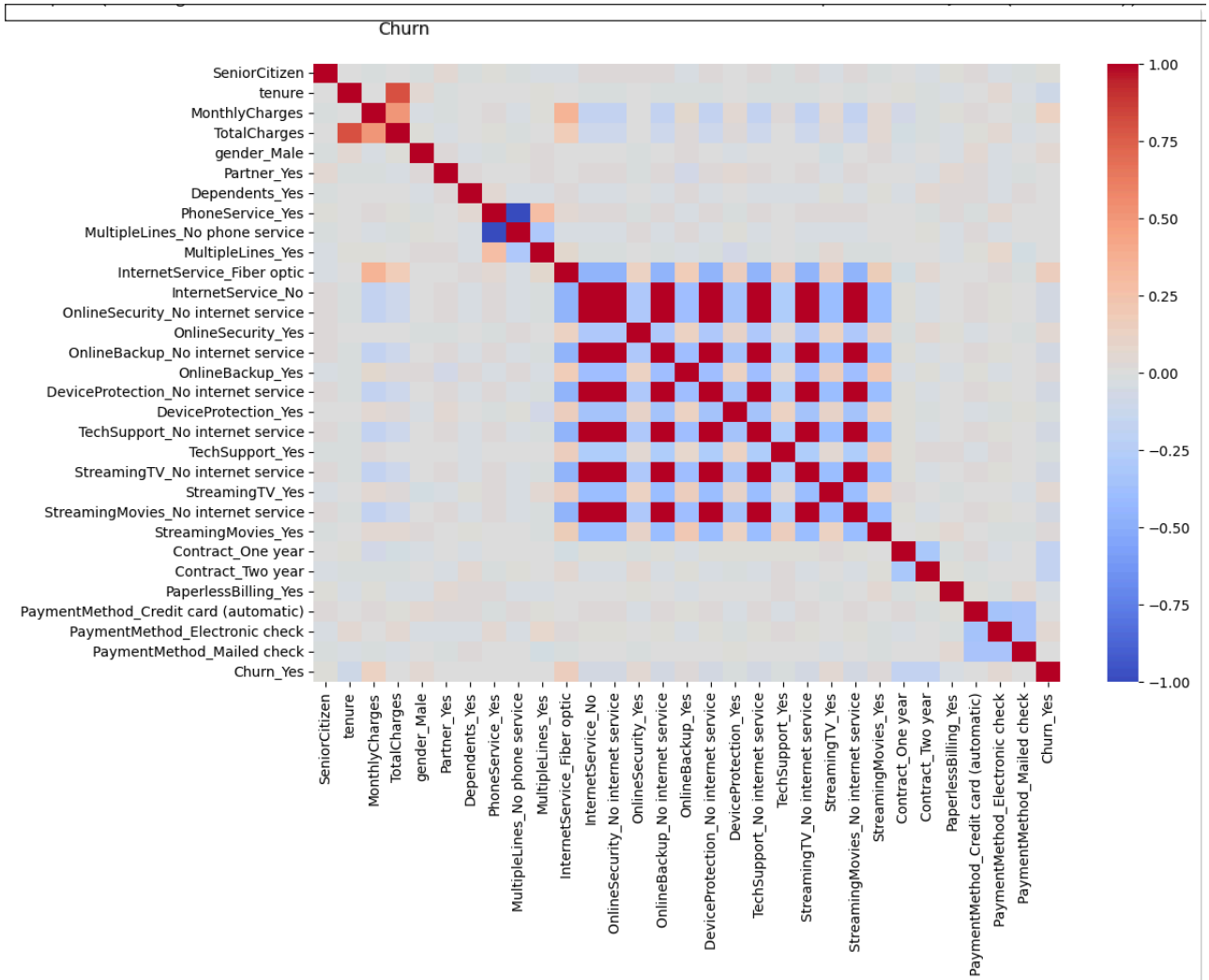
	SeniorCitizen	tenure	MonthlyCharges	TotalCharges	gender_Male	Partner_Yes	Dependents_Yes	PhoneService_Yes	Multip pho
0	0	-1.464748	0.548305	-1.115568	0	0	1	1	
1	0	-0.120590	-1.330931	-0.737304	1	0	0	1	
2	0	0.503483	-1.091309	-0.395599	1	1	0	1	
3	0	-1.560759	0.581201	-1.236431	0	0	0	1	
4	0	-0.408624	0.056594	-0.346696	0	0	0	1	

```
Cleaned shape: (1500, 31)
```

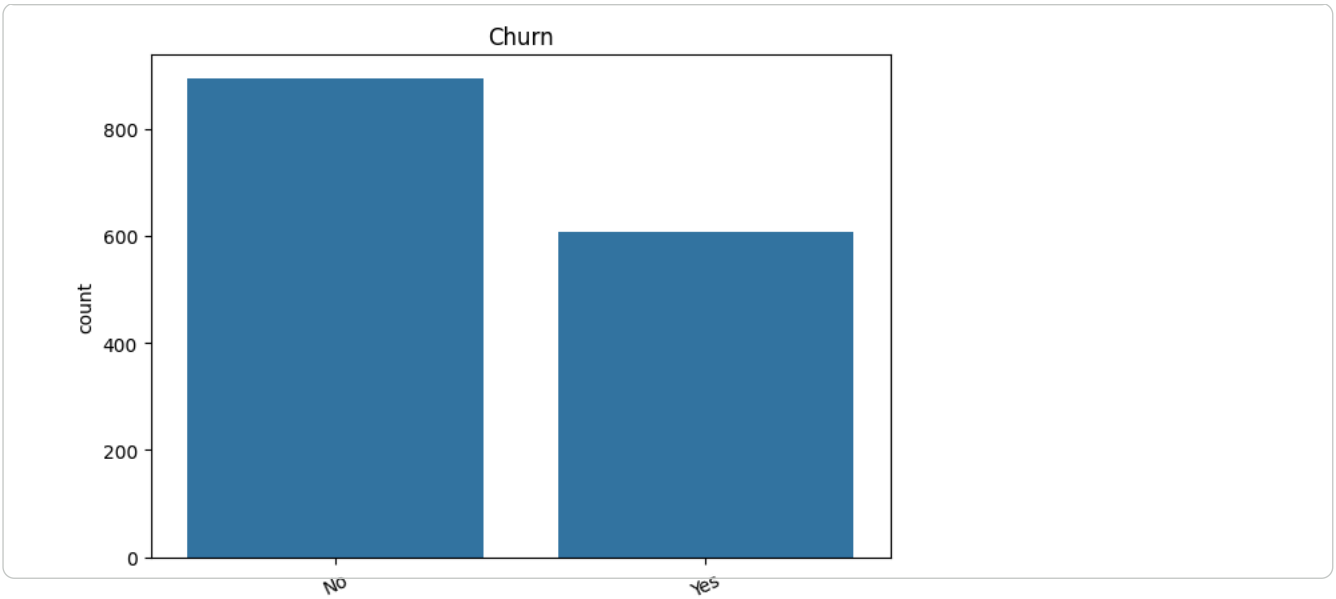


Experiment 2 — Data Discovery and Preparation

```
#Jeyaguru M
df2=telco.copy().drop_duplicates()
if "TotalCharges" in df2.columns:
    df2["TotalCharges"]=pd.to_numeric(df2["TotalCharges"],errors="coerce")
    df2=df2.drop(columns=["customerID"],errors="ignore")
    for c in df2.select_dtypes(include=np.number): df2[c]=df2[c].fillna(df2[c].median())
    for c in df2.select_dtypes(exclude=np.number):
        mode=df2[c].mode(); df2[c]=df2[c].fillna(mode.iloc[0] if len(mode) else "Unknown")
    for c in ["Churn","gender","Contract","PaymentMethod"]:
        if c in df2:
            sns.countplot(data=df2,x=c,hue="Churn" if c!="Churn" else None)
            plt.title(c); plt.xticks(rotation=25); plt.tight_layout(); plt.show()
    print("IQR outlier counts")
    for c in df2.select_dtypes(include=np.number):
        q1,q3=df2[c].quantile([.25,.75]); i=q3-q1
        print(c,((df2[c]<q1-1.5*i)|(df2[c]>q3+1.5*i)).sum())
    d2=pd.get_dummies(df2,drop_first=True,dtype=int)
    X=d2.drop(columns="Churn_Yes"); y=d2["Churn_Yes"]
    from sklearn.model_selection import train_test_split
    X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=.2,random_state=SEED,stratify=y)
    scaler=StandardScaler(); X_train_s=scaler.fit_transform(X_train); X_test_s=scaler.transform(X_test)
    d2.to_csv("CustomerChurn_Preprocessed.csv",index=False)
    print("Train/test:",X_train.shape,X_test.shape,"Saved CustomerChurn_Preprocessed.csv")
else:
    print("Warning: 'telco' DataFrame does not contain Customer Churn data. Columns present are:", list(df2.columns))
```

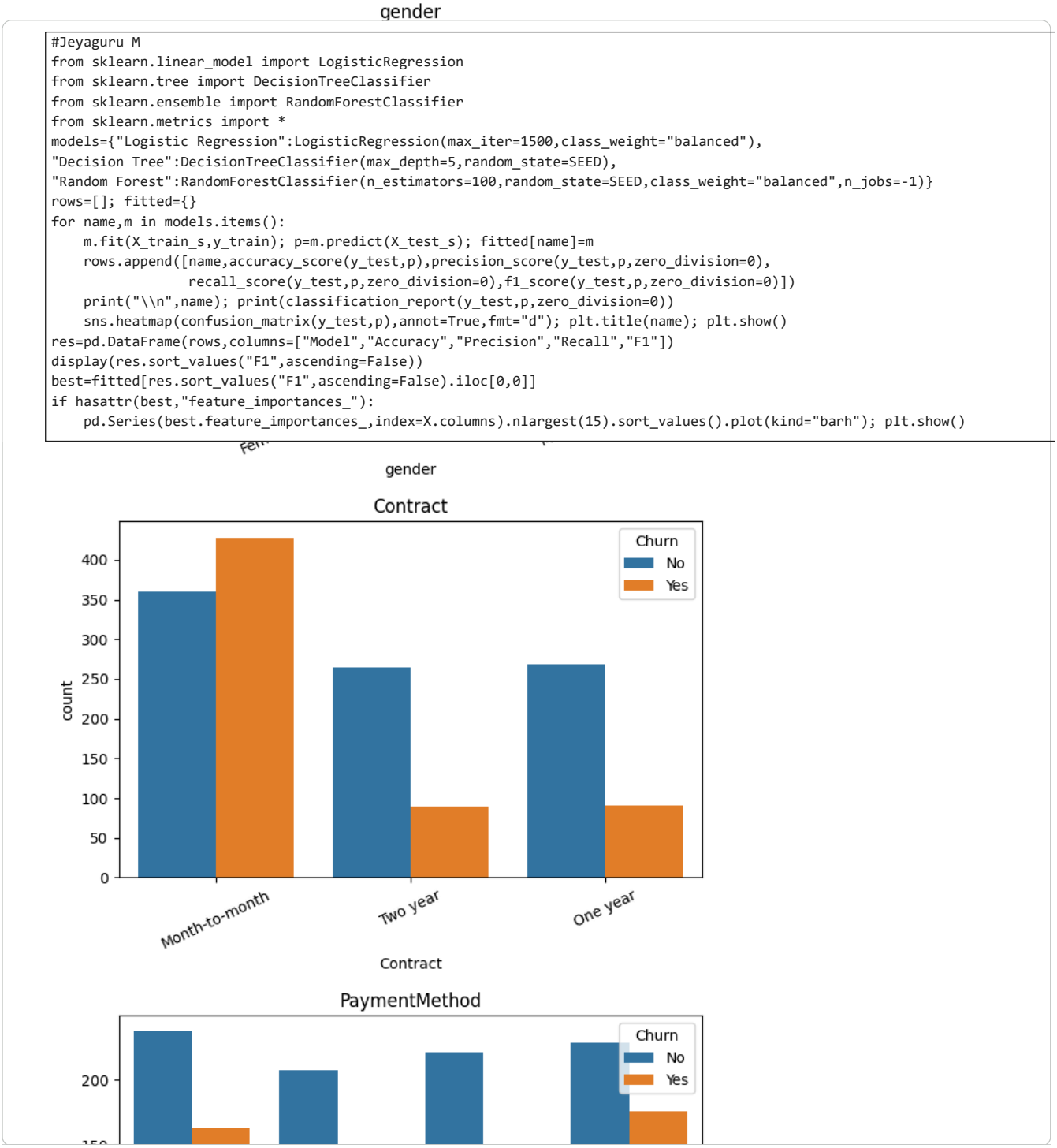


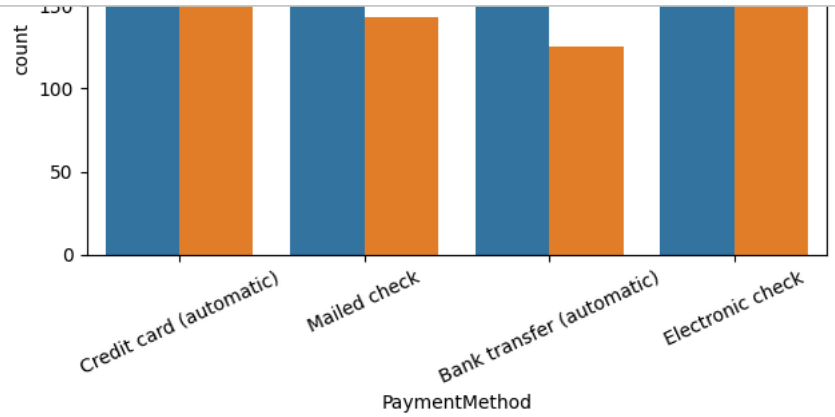

```
Saved Telco_Cleaned_Encoded.csv
```



Experiment 3a — Classification: Logistic Regression, Decision Tree, Random Forest

```
#Jeyaguru M
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import *
models={"Logistic Regression":LogisticRegression(max_iter=1500,class_weight="balanced"),
"Decision Tree":DecisionTreeClassifier(max_depth=5,random_state=SEED),
"Random Forest":RandomForestClassifier(n_estimators=100,random_state=SEED,class_weight="balanced",n_jobs=-1)}
rows=[]; fitted={}
for name,m in models.items():
    m.fit(X_train_s,y_train); p=m.predict(X_test_s); fitted[name]=m
    rows.append([name,accuracy_score(y_test,p),precision_score(y_test,p,zero_division=0),
                 recall_score(y_test,p,zero_division=0),f1_score(y_test,p,zero_division=0)])
    print("\\n",name); print(classification_report(y_test,p,zero_division=0))
    sns.heatmap(confusion_matrix(y_test,p),annot=True,fmt="d"); plt.title(name); plt.show()
res=pd.DataFrame(rows,columns=["Model","Accuracy","Precision","Recall","F1"])
display(res.sort_values("F1",ascending=False))
best=fitted[res.sort_values("F1",ascending=False).iloc[0,0]]
if hasattr(best,"feature_importances_"):
    pd.Series(best.feature_importances_,index=X.columns).nlargest(15).sort_values().plot(kind="barh"); plt.show()
```



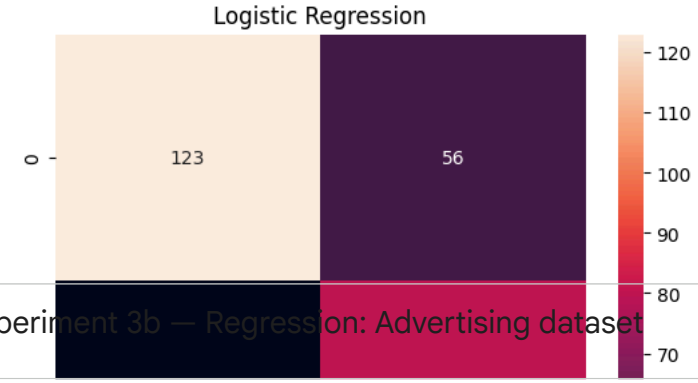



```
IQR outlier counts
SeniorCitizen 264
tenure 0
MonthlyCharges 0
TotalCharges 5
Train/test: (1200, 30) (300, 30) Saved CustomerChurn_Preprocessed.csv
```

```
\n Logistic Regression
              precision    recall  f1-score   support

           0       0.75      0.69      0.72       179
           1       0.59      0.66      0.62       121

    accuracy                           0.68       300
   macro avg       0.67      0.67      0.67       300
weighted avg       0.68      0.68      0.68       300
```



Experiment 3b — Regression: Advertising dataset

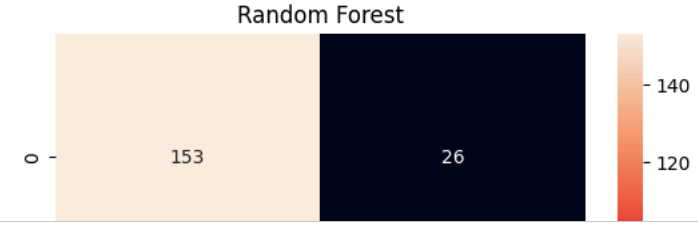
```
#Jeyaguru M
ad_urls=["https://raw.githubusercontent.com/justmarkham/DAT8/master/data/Advertising.csv",
"https://raw.githubusercontent.com/selva86/datasets/master/Advertising.csv"]
ad=None
for u in ad_urls:
    try:
        z=pd.read_csv(u)
        if {"TV","Radio","Newspaper","Sales"}.issubset(z.columns): ad=z; break
    except Exception: pass
if ad is None:
    from google.colab import files
    up=files.upload(); ad=pd.read_csv(next(iter(up)))
ad=ad.loc[:,~ad.columns.str.lower().str.startswith("unnamed")]
display(ad.head())
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error,mean_absolute_error,r2_score
Xa,Xb,ya,yb=train_test_split(ad[["TV","Radio","Newspaper"]],ad.Sales,test_size=.2,random_state=SEED)
poly=PolynomialFeatures(2,include_bias=False); Xap=poly.fit_transform(Xa); Xbp=poly.transform(Xb)
regs={"Linear Regression":(LinearRegression(),Xa,Xb),
"Polynomial Regression":(LinearRegression(),Xap,Xbp),
"Decision Tree Regression":(DecisionTreeRegressor(max_depth=5,random_state=SEED),Xa,Xb)}
rr=[]; preds={}
for n,(m,a,b) in regs.items():
    m.fit(a,ya); p=m.predict(b); preds[n]=p
    rr.append([n,mean_absolute_error(yb,p),mean_squared_error(yb,p),mean_squared_error(yb,p)**.5,r2_score(yb,p)])
display(pd.DataFrame(rr,columns=["Model","MAE","MSE","RMSE","R2"]))
fig,axs=plt.subplots(1,3,figsize=(14,4))
for ax,(n,p) in zip(axs,preds.items()):
    ax.scatter(yb,p); ax.set_title(n); ax.set_xlabel("Actual"); ax.set_ylabel("Predicted")
plt.tight_layout(); plt.show()
```




```
\n Random Forest
              precision    recall  f1-score   support

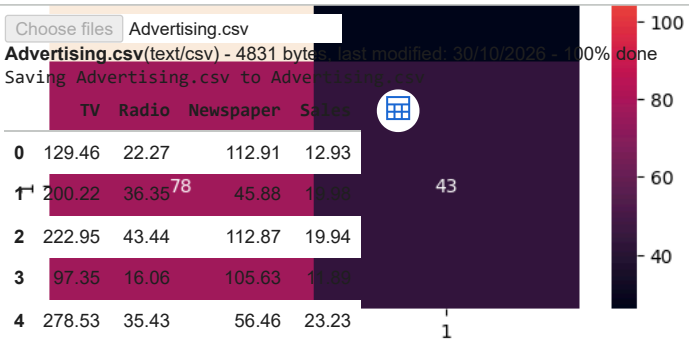
           0       0.66      0.85      0.75       179
           1       0.62      0.36      0.45       121

    accuracy                           0.65       300
   macro avg       0.64      0.61      0.60       300
weighted avg       0.65      0.65      0.63       300
```

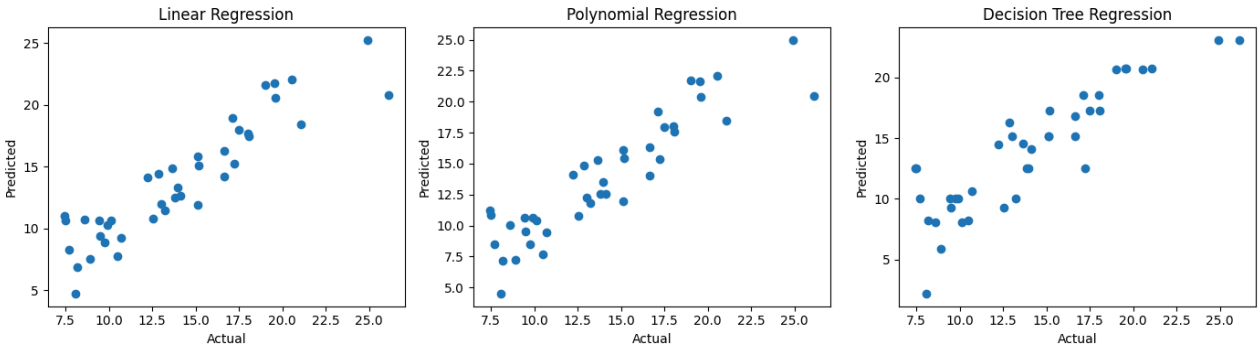


```
Choose files Advertising.csv
Advertising.csv(text/csv) - 4831 bytes, last modified: 30/10/2026 - 100% done
Saving Advertising.csv to Advertising.csv
```

	TV	Radio	Newspaper	Sales
0	129.46	22.27	112.91	12.93
1	200.22	36.35	45.88	19.98
2	222.95	43.44	112.87	19.94
3	97.35	16.06	105.63	16.89
4	278.53	35.43	56.46	23.23



	Model	Accuracy	MAE	MSE	Recall	RMSE	F1	R2
0	Linear Regression	1.576734	3.729183	1.931109	0.834450			
1	Polynomial Regression	1.590488	3.949916	1.987440	0.824651			
1	Decision Tree	0.676667	0.653846	0.421488	0.512563			
2	Decision Tree Regression	1.656610	5.203605	2.281141	0.768996			



Shared: load Reviews.csv for Experiments 4 and 5

```
#Jeyaguru M
if os.path.exists("Reviews.csv"):
    reviews=pd.read_csv("Reviews.csv")
else:
    from google.colab import files
    print("Upload Reviews.csv with a Text column.")
    up=files.upload()
    reviews=pd.read_csv(next(iter(up)))
if "Text" not in reviews: raise ValueError("Reviews.csv must contain a column named Text.")
reviews=reviews[["Text"]].dropna().drop_duplicates().head(10000)
print("Reviews:",len(reviews)); display(reviews.head())
```

```
Reviews: 51
```

	Text
0	The battery life is excellent and lasts nearly...
1	The screen is bright and sharp, but the speake...
2	I received the headphones on time. The sound q...
3	The laptop is fast for programming and browsin...
4	The camera takes beautiful photos in daylight....

Experiment 4 — Information Retrieval Using NLTK (TF-IDF + cosine similarity)

```
#Jeyaguru M
import nltk
for pkg in ["punkt","punkt_tab","stopwords"]: nltk.download(pkg,quiet=True)
from nltk.corpus import stopwords
from nltk.tokenize import wordpunct_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
sw=set(stopwords.words("english"))

def clean_text(s):
    words=wordpunct_tokenize(re.sub(r"[^a-z ]"," ",str(s).lower()))
    return " ".join(w for w in words if w not in sw and len(w)>1)

reviews["clean"]=reviews.Text.astype(str).apply(clean_text)
vec=TfidfVectorizer(max_features=20000)
matrix=vec.fit_transform(reviews.clean)

# We will use the suggested query 'battery life' for the test
query = "battery life"
print(f"Searching for query: '{query}'")

q=vec.transform([clean_text(query)])
scores=cosine_similarity(q,matrix).ravel()

for rank,i in enumerate(scores.argsort()[-5:][::-1],1):
    print(f"\n{rank}. Score={scores[i]:.4f}")
    print(reviews.iloc[i].Text[:1000])
```

```
Searching for query: 'battery life'

1. Score=0.3753
The battery life is excellent and lasts nearly two days with normal use. The phone charges quickly and feels reliable.

2. Score=0.2135
The Bluetooth connection is stable and the battery lasts through a full workday.

3. Score=0.1906
This smartphone has a smooth display, good battery backup, and a responsive fingerprint sensor.

4. Score=0.0000
The textbook covers the syllabus thoroughly and the diagrams make the topics easier to understand.

5. Score=0.0000
The cereal tastes good and is convenient for breakfast, although it contains more sugar than expected.
```

Experiment 5 — Information Retrieval Using spaCy (tokens, lemmas, POS, entities)

```
#Jeyaguru M
!python -m spacy download en_core_web_sm -q
import spacy
from collections import Counter
nlp=spacy.load("en_core_web_sm")
docs=list(nlp.pipe(reviews.Text.astype(str).tolist(),batch_size=100))
if docs:
    print("Tokens:",[t.text for t in docs[0]][:40])
    print("Lemmas:",[t.lemma_ for t in docs[0]][:40])
    print("POS:",[t.pos_ for t in docs[0]][:40])
for pos in ["NOUN","VERB","ADJ","ADV"]:
    counts=Counter(t.lemma_.lower() for d in docs for t in d if t.pos_==pos and not t.is_stop and t.is_alpha)
    print("Frequent",pos,counts.most_common(10))
entities=[(e.text,e.label_) for d in docs for e in d.ents]
display(pd.DataFrame(entities[:30],columns=["Entity","Label"]))
```

```
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
Tokens: ['The', 'battery', 'life', 'is', 'excellent', 'and', 'lasts', 'nearly', 'two', 'days', 'with', 'normal', 'use', '.',
Lemmas: ['the', 'battery', 'life', 'be', 'excellent', 'and', 'last', 'nearly', 'two', 'day', 'with', 'normal', 'use', '.', '
POS: ['DET', 'NOUN', 'NOUN', 'AUX', 'ADJ', 'CCONJ', 'VERB', 'ADV', 'NUM', 'NOUN', 'ADP', 'ADJ', 'NOUN', 'PUNCT', 'DET', 'NOU
Frequent NOUN [('support', 5), ('product', 4), ('delivery', 4), ('battery', 3), ('day', 3), ('picture', 3), ('customer', 3),
Frequent VERB [('work', 5), ('take', 4), ('expect', 4), ('fit', 3), ('arrive', 3), ('keep', 3), ('last', 2), ('feel', 2), ('
Frequent ADJ [('easy', 7), ('good', 5), ('comfortable', 4), ('useful', 4), ('clear', 3), ('smooth', 3), ('accurate', 3), ('s
Frequent ADV [('quickly', 3), ('properly', 2), ('nearly', 1), ('instantly', 1), ('comfortably', 1), ('carefully', 1), ('earl
```

	Entity	Label
0	nearly two days	DATE
1	one week	DATE
2	long hours	TIME
3	Bluetooth	GPE
4	less than five minutes	TIME
5	four days	DATE
6	two	CARDINAL
7	the weekend	DATE
8	Assembly	ORG
9	about thirty minutes	TIME
10	hours	TIME
11	daily	DATE
12	several hours	TIME
13	a month	DATE
14	one	CARDINAL
15	one	CARDINAL
16	two days	DATE

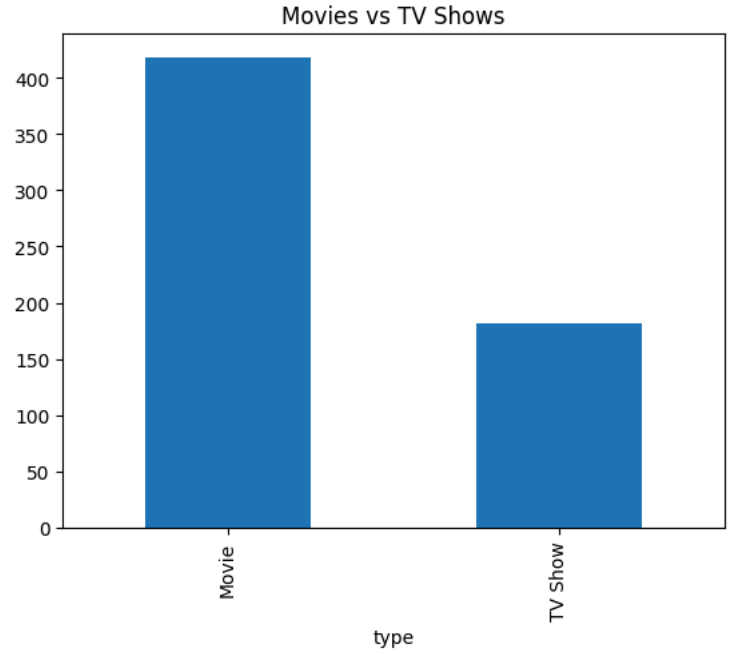
Experiment 6a — Exploratory Data Analysis (Netflix titles)

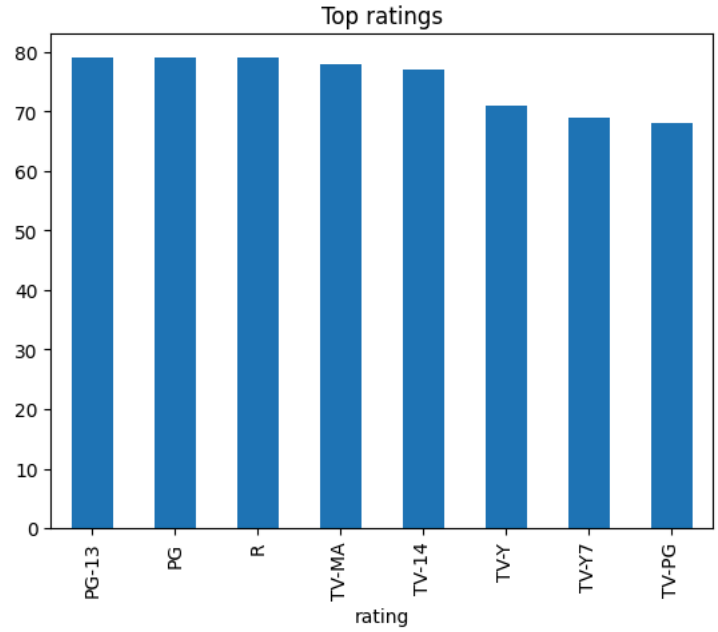
```
#Jeyaguru M
net_urls=["https://raw.githubusercontent.com/shivamb/netflix-shows/master/netflix_titles.csv"]
net=None
for u in net_urls:
    try: net=pd.read_csv(u); break
    except Exception: pass
if net is None:
    from google.colab import files
    up=files.upload(); net=pd.read_csv(next(iter(up)))
print(net.shape); print(net.isna().sum()); print("Duplicates:",net.duplicated().sum())
net["country"]=net.country.fillna("Unknown")
net["date_added"]=pd.to_datetime(net.date_added,errors="coerce")
net["year_added"]=net.date_added.dt.year
net["duration_value"]=pd.to_numeric(net.duration.astype(str).str.extract(r"(\\d+)")[0],errors="coerce")
net.type.value_counts().plot(kind="bar",title="Movies vs TV Shows"); plt.show()
net.rating.value_counts().head(10).plot(kind="bar",title="Top ratings"); plt.show()
net.release_year.dropna().plot(kind="hist",bins=25,title="Release year"); plt.show()
net.year_added.value_counts().sort_index().plot(marker="o",title="Titles added by year"); plt.show()
net.country.str.split(", ").explode().value_counts().head(10).sort_values().plot(kind="barh",title="Top countries"); plt.sho
net.listed_in.dropna().str.split(", ").explode().value_counts().head(12).sort_values().plot(kind="barh",title="Top genres");
display(net[["type","rating","duration_value","release_year"]].describe(include="all"))
```

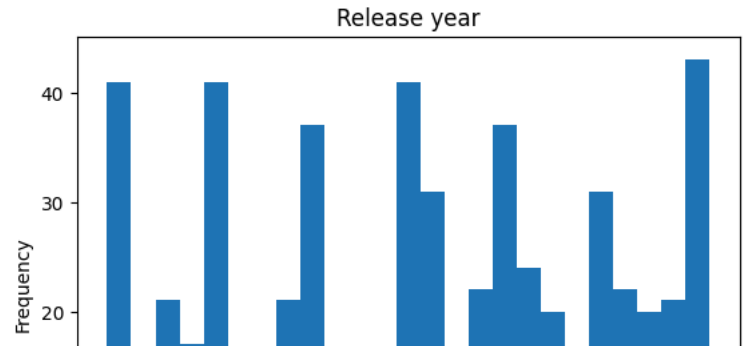
Choose files netflix_titles.csv

netflix_titles.csv(text/csv) - 115307 bytes, last modified: 30/10/2026 - 100% done

```
Saving netflix_titles.csv to netflix_titles.csv
(600, 12)
show_id           0
type              0
title             0
director        112
cast            156
country           0
date_added        0
release_year      0
rating            0
duration          0
listed_in         0
description       0
dtype: int64
Duplicates: 0
```








10